



ÉCOLE
POLYTECHNIQUE
DE BRUXELLES

Projet CPA : arbre phylogénétique

LECOMTE Anatole

MOUSSIAUX Hugo

SAMADOV Komildzhon

Prof. ROLAND Jérémie

2024

A large, faint, light blue watermark of the University of Brussels seal is visible in the background. It features a circular border with the Latin motto 'SCIENTIA VINCERE TE' and a central emblem depicting a sunburst above a shield with various symbols.

Contents

1	Choix de conception	1
1.1	Problème initial	1
1.2	Choix de structure de données	2
1.2.1	Choix initial	2
1.2.2	Choix final	3
2	Classes	4
3	Structure du code	5
3.1	Implémentation	5
3.2	Optimisation	5
3.3	Complexité	6

1.1 Problème initial

Pour ce projet, le cahier des charges impose la lecture d'une matrice de dissimilitude venant d'un fichier texte, il faut donc trouver une structure de données adéquate pour la stocker. L'objectif est de trouver une structure de données qui permet de minimiser la complexité du programme car celui-ci sera amené à travailler avec des matrices de dimensions très importantes. La littérature ("MGUPGMA: A Fast UPGMA Algorithm With Multiple Graphics Processing Units Using NCCL", Guan-Jie Hua, Che-Lun Hung, Chun-Yuan Lin, Fu-Che Wu, Yu-Wei Chan and Chuan Yi Tang) nous apprend que l'algorithme le plus intuitif possède une complexité en $O(n^3)$ mais qu'il est possible de résoudre le problème en $O(n^2)$ grâce des algorithmes très optimisés. Afin de pouvoir évaluer la complexité du programme avant son implémentation et ainsi optimiser le choix de structure de données, les différentes étapes de résolution sont listées ci-dessous :

- Lecture et insertion des éléments de la matrice (à effectuer une seule fois)
- Boucle à répéter (n-1) fois:
 - Recherche de la valeur minimale
 - Création d'un ancêtre et calcul de ses valeurs
 - Insertion de ces nouvelles valeurs dans la structure de données
 - Suppression des valeurs correspondant aux deux descendants issus de cet ancêtre

Le calcul de complexité est réalisé sur base du tableau ci-après :

	INSERT	DELETE	SEARCH	MINIMUM	MAXIMUM	PREDECESSOR	SUCCESSOR
Tableau	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Tableau trié	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Liste simplement chaînée	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Liste doublement chaînée	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Liste simplement chaînée triée	$O(n)$	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Liste doublement chaînée triée	$O(n)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Tas-max	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Arbre binaire de recherche (équilibré)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

Figure 1.1: Complexité associée aux opérations de base dans les structures de données

1.2 Choix de structure de données

Lors de la réalisation du projet, deux structures de données distinctes ont été envisagées pour la conception du programme. Chacune de ces structures présente des avantages notables, mais aussi des inconvénients à prendre en compte. Cette section se propose d'analyser les deux choix, afin de mieux comprendre leurs impacts sur la performance et leurs limitations.

1.2.1 Choix initial

La structure de données qui a été initialement retenue est l'arbre binaire de recherche (équilibré). Les deux sous-sections qui suivent abordent, respectivement, les raisons justifiant un tel choix et son implémentation.

Motivations

La première étape consistant à lire un nombre d'éléments de l'ordre de $\frac{n^2}{2}$ et à ensuite les insérer dans la structure de données, il est impératif d'avoir un coût d'insertion inférieur à $O(n)$ pour ne pas dépasser une complexité globale de $O(n^2 \log(n))$. Cette première restriction permet déjà d'éliminer les listes chaînées triées et le tableau trié à partir du tableau 1.1.

Il faut ensuite effectuer les $(n-1)$ boucles en réalisant, à chaque itération, une recherche de minimum, $2(n'-1)$ recherches d'éléments, $(n'-2)$ insertions, $(2(n'-1)-1)$ suppressions¹. En consultant à nouveau le tableau 1.1, on constate que le tas-min (non représenté dans le tableau mais dont les complexités sont identiques au tas-max à l'exception des recherches de minimum et de maximum qui sont permutées) et l'arbre binaire de recherche (équilibré) sont les deux structures de données les plus avantageuses en raison de leur coût pour la recherche de minimum. Enfin, le choix de l'arbre binaire équilibré plutôt que du tas-min repose sur le fait qu'une approche algorithmique avait déjà été établie lors de l'évaluation de la complexité.

Implémentation

L'arbre est constitué de noeuds possédant chacun les attributs suivants : une valeur, trois pointeurs (pour la structure d'arbre binaire) et deux objets animaux correspondant au noeud considéré. Les objets animaux,

¹Remarque : n' représente la taille de la matrice à l'itération considérée (matrice $n' \times n'$)

quant à eux, possèdent chacun une liste de pointeurs vers tous les noeuds dans lesquels ils sont impliqués. Après avoir trouvé le minimum, les valeurs du nouvel ancêtre sont calculées grâce aux listes de pointeurs possédées par les objets animaux. Il reste alors à créer les noeuds de l'ancêtre et les insérer. Finalement, les noeuds liés aux deux animaux fusionnés lors de la création du nouvel ancêtre sont supprimés. Malheureusement, cette implémentation a conduit à une impasse lors de l'actualisation des listes de noeuds dans les objets animaux qu'il n'a pas été possible de résoudre. Pour information, le code correspondant est tout de même disponible dans le GitLab du projet, dans le dossier "versions antérieures".

1.2.2 Choix final

La structure de données utilisée dans la version finale du projet est le vector de vector. Similairement à l'arbre binaire de recherche, les motivations sont développées ci-dessous. L'implémentation est, quant à elle, abordée en détails dans les chapitres 2 et 3.

Motivations

Tout comme l'arbre binaire, la création initiale du vector de vector s'effectue avec une complexité inférieure à $O(n^3)$. En effet, dans ce dernier, les insertions se font avec une complexité en $O(1)$ en raison du caractère non trié de cette structure de données.

Concernant les $(n-1)$ boucles, la recherche de minimum s'effectuerait logiquement avec une complexité en $O((n')^2)$ (où (n') est la dimension de la matrice à l'itération considérée) car cette structure de données n'est pas triée. Cependant, des optimisations présentées plus loin dans le rapport permettent, en réalité, de réduire le temps de recherche du minimum.

Deux avantages non négligeables de la structure considérée sont d'une part un besoin en stockage drastiquement réduit et d'autre part une simplification de la structure du code. L'occupation limitée de la mémoire s'explique notamment par la simplicité du vector de vector qui, contrairement à un arbre ou à une liste chaînée, ne nécessite pas de stocker des pointeurs vers d'autres noeuds. De plus, la recherche d'éléments est grandement simplifiée car ceux-ci sont directement accessibles via la paire d'indices qui leur est associée. Une fois le minimum trouvé (et donc sa paire d'indices également), le calcul des valeurs du nouvel ancêtre est facilité car l'accès aux valeurs nécessaires est immédiat. Enfin, les suppressions et insertions se font facilement car l'insertion a lieu en fin de vector et la suppression nécessite uniquement l'indice de l'élément à supprimer. Il convient cependant de noter que la suppression peut engendrer une augmentation importante du coût, cet aspect sera développé dans la section complexité du chapitre 3.

Le choix des classes dans le code actuel repose sur une conception qui vise à encapsuler les responsabilités et à simplifier la gestion des données dans le cadre de la construction d'un arbre phylogénétique. Deux classes principales, Ligne et Matrice, permettent de répondre efficacement aux besoins spécifiques de l'algorithme.

La classe Ligne encapsule toutes les opérations liées à une ligne individuelle de la matrice. Chaque instance de cette classe représente une espèce, caractérisée par son nom, un vector contenant les indices de dissimilitude, une valeur étant l'indice de dissimilitude minimal ainsi que sa position dans le vector. Ce choix permet de centraliser des fonctionnalités essentielles, comme la gestion des valeurs associées à chaque ligne, la recherche de la distance minimale, ou encore l'ajout et la suppression de valeurs. L'utilisation d'une classe dédiée pour ce type de données favorise une séparation claire entre les opérations sur les lignes et celles sur la matrice globale.

La classe Matrice, de son côté, est conçue pour s'occuper des interactions entre les lignes et exécuter les étapes clés de l'algorithme pour calculer l'arbre phylogénétique. Elle représente la matrice de distances dans son ensemble, mais délègue aux instances de Ligne la gestion des données. Ce choix permet à la classe Matrice de se concentrer sur la fusion des espèces pour construire l'arbre phylogénétique, la gestion de la mémoire lors de la suppression de lignes ou de colonnes, et le calcul des distances entre un nouvel ancêtre et les autres espèces restantes. Grâce à cette répartition des responsabilités, le code gagne en clarté. L'intérêt principal de cette structure par classes réside dans la capacité à facilement modifier le code ainsi qu'à sa bonne structure. La classe Ligne agit comme un conteneur structuré pour les données de chaque espèce, tandis que la classe Matrice se concentre sur la logique globale et les interactions entre les lignes.

3.1 Implémentation

Dans un premier temps, la matrice du fichier va être convertie en un vector de pointeurs vers des objets d'une classe "Ligne". L'initialisation des attributs des Lignes se fait dès la lecture de la matrice issue du fichier texte. Il faut ensuite réaliser la boucle jusqu'à trouver l'unique ancêtre. Le minimum est déterminé sur base d'un balayage des valeurs minimales pré-sauvegardées de chaque Ligne. L'indice de ce minimum dans la Ligne à laquelle il appartient ainsi que l'indice de la Ligne dans la Matrice permettent d'accéder aux valeurs nécessaires aux calculs des valeurs du nouvel ancêtre (qui est une Ligne). Une fois les valeurs calculées, il faut supprimer les lignes et colonnes des deux descendants fusionnés tout en re-cherchant le minimum des lignes pour lesquelles le minimum a été emporté lors de la suppression. Enfin, la Ligne ancêtre est ajoutée à la fin du vector Matrice. L'itération suivante peut alors commencer. Pour mieux comprendre le fonctionnement générale du programme, un pseudo-code est disponible 3.2.

3.2 Optimisation

Puisque le programme doit traiter des matrices de tailles allant jusqu'à 1000, voire 10 000 espèces, il était important d'optimiser le temps d'exécution. Dans cette optique, plusieurs optimisations ont été implémentées pour réduire significativement le nombre d'opérations effectuées au cours des itérations. Ces améliorations permettent d'accélérer les calculs. Les trois principales intégrations réalisées dans le programme sont les suivantes :

Premièrement, la création de la matrice et la recherche du minimum de chaque Ligne ont été effectuées simultanément. Dès la lecture des données issues du fichier texte, les valeurs numériques sont ajoutées aux objets de la classe Ligne. Dans le même temps, une comparaison des valeurs entre elles permet d'identifier la distance minimale de chaque ligne. Cette approche présente deux avantages majeurs : elle élimine la nécessité de parcourir toutes les valeurs de chaque Ligne pour trouver les minima et permet de disposer, dès le début, des informations nécessaires pour les itérations suivantes. Ainsi, le coût de calcul du minimum global est réduit puisqu'il suffit de comparer les minima de chaque Ligne.

Deuxièmement, les minima de chaque Ligne sont sauvegardés entre les itérations. En effet, lors de la suppression des deux colonnes correspondant aux espèces fusionnées, si les minima de chaque ligne ne sont pas supprimés dans le processus, ils peuvent alors être réutilisés, ce qui évite de recalculer ces valeurs à chaque itération. Si les minima ont été supprimés, une mise à jour est réalisée : les valeurs sont à nouveau parcourues pour les lignes concernées, et un nouveau minimum est déterminé. Cette stratégie permet de réduire drastiquement la complexité du programme en diminuant le nombre de calculs redondants. Avec ce système, lors de l'ajout de l'ancêtre à la matrice, il est uniquement nécessaire de comparer les nouvelles valeurs calculées avec les minima de chaque Ligne.

Troisièmement, lors de la création de l'ancêtre, les calculs des nouvelles distances et la recherche du minimum de cette Ligne se font en même temps, ce qui permet de ne pas parcourir tout le vecteur une deuxième fois. Ces optimisations ont permis de réduire le temps de calcul pour une matrice de taille 1050 par 4. Le pseudo-code ci-dessous permet de mieux comprendre le fonctionnement de la fonction *supprimer_colonne*.

```
Procedure supprimer_colonne(ind_ligne_min, ind_colonne_min, Matrice)
  for Ligne in Matrice do
    Delete Ligne—>valeurs[ind_colonne_min]
    Delete Ligne—>valeurs[ind_colonne_min]
    if (ind_ligne_min or ind_colonne_min) == Ligne—>indice_valeur_minimale do
      valeur_min =  $+\infty$ 
      indice_position = -1
      for i = 0 upto size(Ligne—>valeurs) do
        If Ligne—>valeurs[i] < valeur_min
          valeur_min = Ligne—>valeurs[i]
          indice_position = i
      Ligne—>valeur_minimale = valeur_min
      Ligne—>indice_valeur_minimale = indice_position
```

Figure 3.1: Pseudo-code de la fonction optimisée *supprimer_colonne*

3.3 Complexité

Pour bien comprendre le coût du programme, il faut faire une analyse de sa complexité. La fonction *lecture_matrice* présente une complexité de $O(n^2)$, car elle parcourt environ $\frac{n^2}{2}$ éléments dans une matrice triangulaire inférieure et l'insertion se fait en $O(1)$. La fonction principale *main* repose sur une boucle while qui s'exécute tant que la taille de la matrice est supérieure à 1. À chaque itération, la recherche du minimum global via *calcul_minimum* est réalisée en $O(n)$ puisqu'il est uniquement utile de comparer les minima de chaque Ligne entre-eux. Les suppressions de lignes et colonnes affichent respectivement des complexités de $O(n)$ et $O(n^2)$. Pour la suppression des éléments des lignes, une fois le pointeur d'une Ligne supprimé,

il faut décaler tout ceux qui le suivent. Pour la suppression des éléments des colonnes, la complexité est en $O(n)$ pour la suppression car il est nécessaire de rentrer dans chaque Ligne pour supprimer le bon élément, et en $O(n)$ pour le déplacement des éléments restant pour chaque ligne, ce qui amène une complexité de $O(n^2)$. Le calcul des valeurs du nouvel ancêtre est linéaire, soit $O(n)$. En combinant ces facteurs, la complexité totale du programme pour la construction de l'ancêtre final s'élève à $O(n^3)$.

```

Procedure lecture_matrice(fichier)
  for ligne_texte in fichier
    valeur_min =  $+\infty$ 
    indice_min = -1
    Create an object Ligne
    Get nom in ligne_texte
    for i = 0 upto n do
      Add indice_dissimilitude to Ligne—>valeurs
      If Ligne—>valeurs[i] < valeur_min
        valeur_min = Ligne—>valeurs[i]
        indice_min = i
      Ligne—>valeur_minimale = valeur_min
      Ligne—>indice_valeur_minimale = indice_min
      Matrice.pushback(Ligne)

// Fonction principale de l'algorithme
Procedure main(fichier)
  Matrice = lecture_matrice(fichier)
  while size(Matrice) > 1 do
    indice_ligne_minimale, indice_colonne_minimale = calcul_minimum(Matrice)
    ancetre = creer_nouvel_ancetre(indice_ligne_minimale, indice_colonne_minimale)
    ancetre—>valeurs = calcul_valeur(indice_ligne_minimale, indice_colonne_minimale)
    supprimer_ligne(indice_ligne_minimale, indice_colonne_minimale)
    supprimer_colonne(indice_ligne_minimale, indice_colonne_minimale, Matrice)
    Matrice.pushback(ancetre)
  Print Matrice(0)—>nom
  Return 0

Procedure calcul_minimum(Matrice)
  minimum =  $+\infty$ 
  valeur_min_ligne =  $+\infty$ 
  indice_ligne_minimale = -1
  indice_colonne_minimale = -1
  for i = 0 upto size(Matrice) do
    valeur_min_ligne = Matrice[i]—>valeur_minimale
    if valeur_min_ligne < minimum
      minimum = valeur_min_ligne
      indice_ligne_minimale = i
      indice_colonne_minimale = Matrice[i]—>indice_valeur_minimale
  Return indice_ligne_minimale, indice_colonne_minimale

```

$O(n^2)$
 $O(n)$
 $O(n)$
 $O(1)$
 $O(n)$
 $O(n)$
 $O(n^2)$
 $O(1)$

Figure 3.2: Pseudo-code du programme